In [110…

```
#Netflix, founded on August 29, 1997, in Scotts Valley, California, by Reed Hastings and Marc Randolph,
#began as a DVD-by-mail service. Hastings, a computer scientist and mathematician,
#co-founded the company after being inspired by a hefty late fee from a traditional video rental store. Randolph,
#a marketing executive, played a pivotal role in shaping Netflix's early user interface and branding.


#What is the most frequent genre of movies released on Netflix?
#Which has highest votes in vote avg column?
#What movie got the highest popularity? what's its genre?
#What movie got the lowest popularity? what's its genre?
#Which year has the most filmmed movies?
```

In [88]:

```
# Import libraries used for data analysis and visualization

import pandas as pd          # Pandas → used for data manipulation and working with DataFrames
import numpy as np           # NumPy → used for numerical operations and arrays
import matplotlib.pyplot as plt   # Matplotlib → basic library for creating plots and charts
import seaborn as sns        # Seaborn → advanced statistical data visualization built on matplotlib
```

In [89]:

```
# Read the CSV file from the given path and store it in a DataFrame
df = pd.read_csv(r"E:\new drive\Netflix.csv")
```

In [90]:

```
# Show first 5 rows
df.head()
```

Out[90]:

	Release_Date	Title	Overview	Popularity	Vote_Count	Vote_Average	Original_Language	Genre	
0	12/15/2021	Spider-Man: No Way Home	Peter Parker is unmasked and no longer able to...	5083.954	8940	8.3	en	Action, Adventure, Science Fiction	https://image.tmdb.org/t/p/o
1	3/1/2022	The Batman	In his second year of fighting crime, Batman u...	3827.658	1151	8.1	en	Crime, Mystery, Thriller	https://image.tmdb.org/t/p/o
2	2/25/2022	No Exit	Stranded at a rest stop in the mountains durin...	2618.087	122	6.3	en	Thriller	https://image.tmdb.org/t/p/orig
3	11/24/2021	Encanto	The tale of an extraordinary family, the Madri...	2402.201	5076	7.7	en	Animation, Comedy, Family, Fantasy	https://image.tmdb.org/t/p/ori
4	12/22/2021	The King's Man	As a collection of history's worst tyrants and...	1895.511	1793	7	en	Action, Adventure, Thriller, War	https://image.tmdb.org/t/p/ori

In [91]:

```
# Check dataset shape (rows, columns)
df.shape
```

Out[91]: (9837, 9)

In [92]:

```
# Check data types and null values
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9837 entries, 0 to 9836
Data columns (total 9 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Release_Date       9837 non-null   object
 1   Title              9828 non-null   object
 2   Overview           9828 non-null   object
 3   Popularity         9827 non-null   float64
 4   Vote_Count         9827 non-null   object
 5   Vote_Average       9827 non-null   object
 6   Original_Language  9827 non-null   object
 7   Genre              9826 non-null   object
 8   Poster_Url         9826 non-null   object
dtypes: float64(1), object(8)
memory usage: 691.8+ KB
```

In [93]:

```
# Count missing values
```

```
df.isnull().sum()
```

Out[93]:
```
Release_Date          0
Title                 9
Overview              9
Popularity           10
Vote_Count           10
Vote_Average         10
Original_Language    10
Genre                11
Poster_Url           11
dtype: int64
```

In [94]:
```
# Remove unnecessary columns
df = df.drop(columns=["Overview","Original_Language","Poster_Url"], errors="ignore")
```

In [95]:
```
# Remove spaces after commas in Genre column
df["Genre"] = df["Genre"].str.replace(", ", ",")
```

In [96]:
```
# Convert Release_Date to only year
df["Release_Date"] = pd.to_datetime(df["Release_Date"], errors="coerce").dt.year.astype("Int64")
```

In [97]:
```
# Check data types after cleaning
df.dtypes
```

Out[97]:
```
Release_Date      Int64
Title            object
Popularity      float64
Vote_Count       object
Vote_Average     object
Genre            object
dtype: object
```

In [98]:
```
# Convert Vote_Average column to numeric
df["Vote_Average"] = pd.to_numeric(df["Vote_Average"], errors="coerce")
```

In [99]:
```
# Create rating labels based on vote average
df["Rating_Label"] = pd.cut(
    df["Vote_Average"],
    bins=[0, 6.9, 7.9, 10],
    labels=["Low", "Good", "Excellent"]
)
```

In [100…
```
# Remove Vote_Average column after creating labels
df=df.drop(columns=["Vote_Average"],errors="ignore")
```

In [101…
```
# Preview dataset
df.head()
```

Out[101…

	Release_Date	Title	Popularity	Vote_Count	Genre	Rating_Label
0	2021	Spider-Man: No Way Home	5083.954	8940	Action,Adventure,Science Fiction	Excellent
1	2022	The Batman	3827.658	1151	Crime,Mystery,Thriller	Excellent
2	2022	No Exit	2618.087	122	Thriller	Low
3	2021	Encanto	2402.201	5076	Animation,Comedy,Family,Fantasy	Good
4	2021	The King's Man	1895.511	1793	Action,Adventure,Thriller,War	Good

In [102…
```
# Split multiple genres into list
# Convert multiple genres into separate rows
df["Genre"] = df["Genre"].str.split(",")
df = df.explode("Genre")
```

In [103…
```
# Check new dataset shape
df.shape
```

Out[103…
```
(25803, 6)
```

In [104…
```
# Count unique values in each column
df.nunique()
```

Out[104…
```
Release_Date     102
Title           9514
Popularity      8160
Vote_Count      3267
Genre             19
Rating_Label       3
dtype: int64
```

In [105…
```
# Genre statistics
df["Genre"].describe()
```

Out[105…

```
count     25792
unique       19
top       Drama
freq       3744
Name: Genre, dtype: object
```

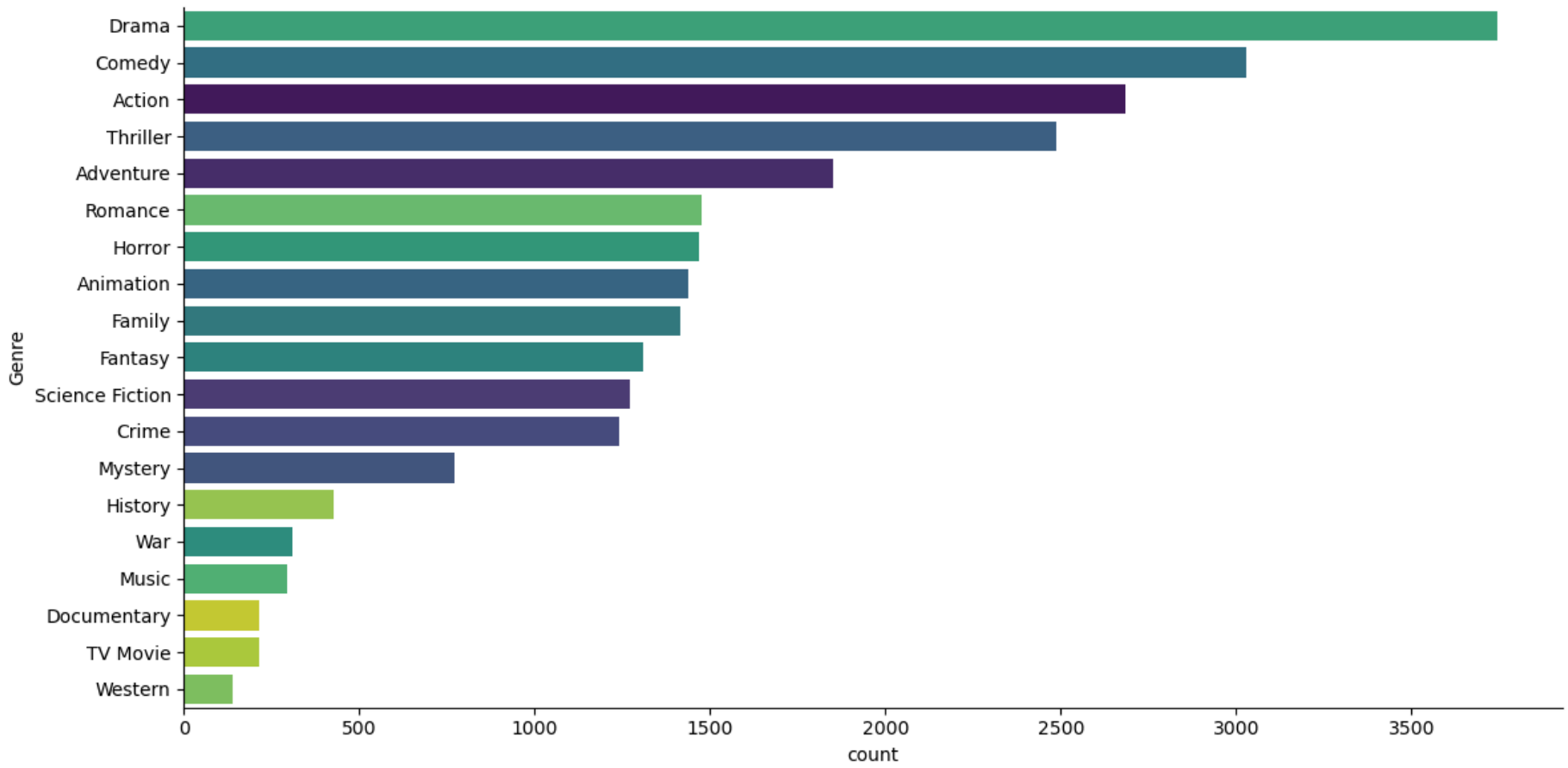
In [106…

```
# Plot genre frequency (largest to smallest)
df = df.reset_index(drop=True)

sns.catplot(
    y="Genre",
    hue="Genre",
    data=df,
    kind="count",
    palette="viridis",
    legend=False,
    height=6,
    aspect=2,
    order=df["Genre"].value_counts().index
)
```

Out[106…

```
<seaborn.axisgrid.FacetGrid at 0x1cf811f1810>
```



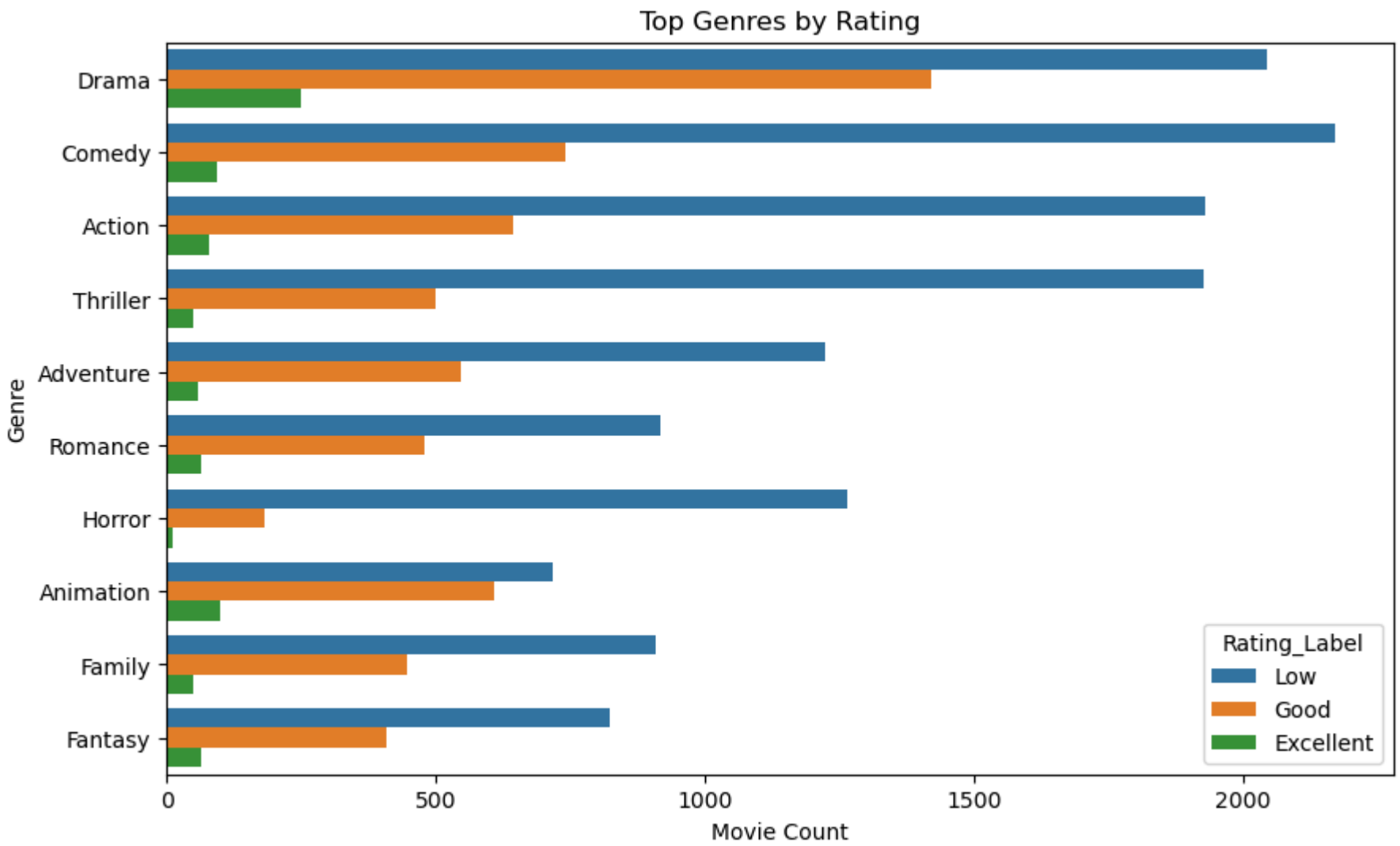
In [107…

```
# Plot top 10 genres by rating category
df = df.reset_index(drop=True)

plt.figure(figsize=(10,6))

sns.countplot(
    y="Genre",
    hue="Rating_Label",
    data=df,
    order=df["Genre"].value_counts().head(10).index
)

plt.title("Top Genres by Rating")
plt.xlabel("Movie Count")
plt.ylabel("Genre")
plt.show()
```



In [109…

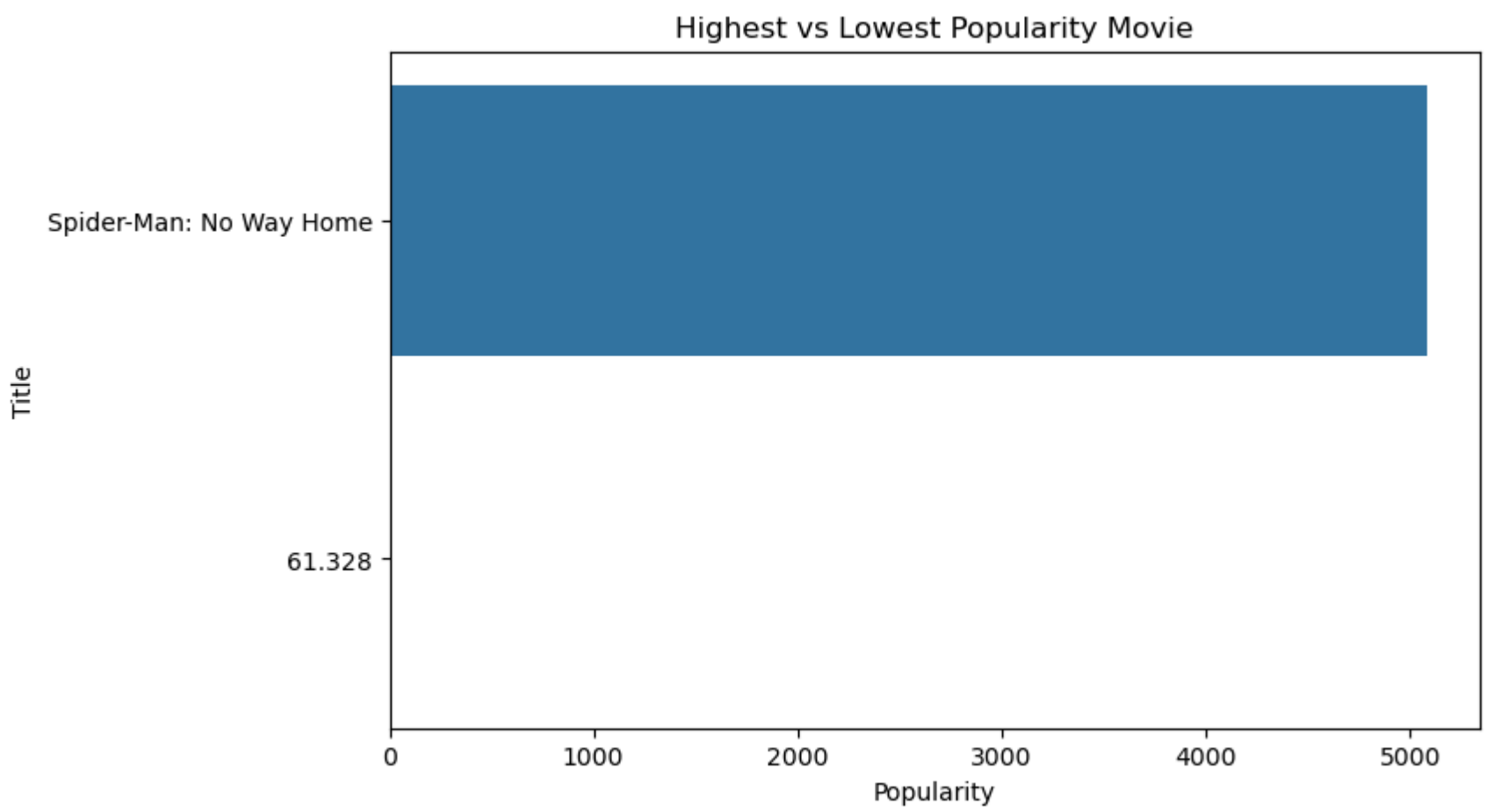
```
# get highest and lowest popularity movies
highest = df.loc[df["Popularity"].idxmax(), ["Title","Popularity"]]
lowest = df.loc[df["Popularity"].idxmin(), ["Title","Popularity"]]

# create dataframe for plotting
pop_df = pd.DataFrame([highest, lowest])

# plot bar chart
plt.figure(figsize=(8,5))
sns.barplot(x="Popularity", y="Title", data=pop_df)

# add title
plt.title("Highest vs Lowest Popularity Movie")

plt.show()
```



Summary In this project, I analyzed a Netflix movies dataset using Python. First, I cleaned the data by removing unnecessary columns and converting the release date into year format. Then I processed the genre column by splitting multiple genres so each genre could be analyzed separately. After cleaning the data, I performed exploratory data analysis to understand movie trends on Netflix. I found the most frequent movie genre, the movie with the highest popularity, and the movie with the lowest popularity along with their genres. I also identified which year had the most movies released. To better understand the data, I created visualizations such as genre distribution charts and top genres by rating category using Seaborn and Matplotlib. Overall, this analysis helped reveal patterns in Netflix movies, including which genres appear most often and how popularity and ratings vary across movies.

In [ ]: